

A Locally-Executed Triage Pipeline for Roman Urdu Emergency Complaints

Learned stop-word removal, offline neural translation, and a deterministic anatomical safety gate for a four-level cardiac triage classifier

ML_Predictor · Research prototype

Model bundle `triage_model_embedding_english/` · trained 9 September 2026

ABSTRACT

This chapter describes how the system was built and why each component was chosen. The task is to take a patient complaint written in Roman Urdu — Urdu typed in the Latin alphabet, as it is actually written on a phone in a Pakistani emergency department — together with the patient's vital signs, and return a triage level from 1 (most urgent) to 4 (least urgent).

Three design commitments shape everything that follows. First, every stage of inference runs on the clinician's own machine; no complaint text leaves it. Second, the stop-word list is *learned* from labelled data rather than hand-written, because no published Roman Urdu stop-word list exists and an ad-hoc one cannot be reproduced by anyone else. Third, the system is permitted to refuse. When its internal checks fail it returns no triage level at all, on the view that in this setting a withheld answer is safer than a confident wrong one.

We report an ablation across two embedding configurations, the deployed model's performance on 451 held-out cases, and — at some length — the approaches we tried and abandoned, because several of those negative results informed the final architecture. The embedding comparison is reported in §3.9.

3.1 System overview

The system is a pipeline, not a single model. A complaint entered in Roman Urdu passes through six stages before a triage level is produced, and it can be stopped at three of them. Figure 1 shows the whole path.

It is worth being explicit about why a pipeline rather than an end-to-end model. An end-to-end classifier trained directly on Roman Urdu text would be simpler to describe and simpler to train. It would also be opaque at exactly the moment it matters: when a clinician asks why a complaint was rated Level 1, "the network decided" is not a usable answer. Every stage in Figure 1 produces an intermediate artefact a human can read — the English translation, the normalised token string, the list of removed stop words, the final feature vector — and the interface displays all of them. The cost is a longer pipeline with more places to fail. We considered that an acceptable trade for a system whose output a person has to be willing to act on.

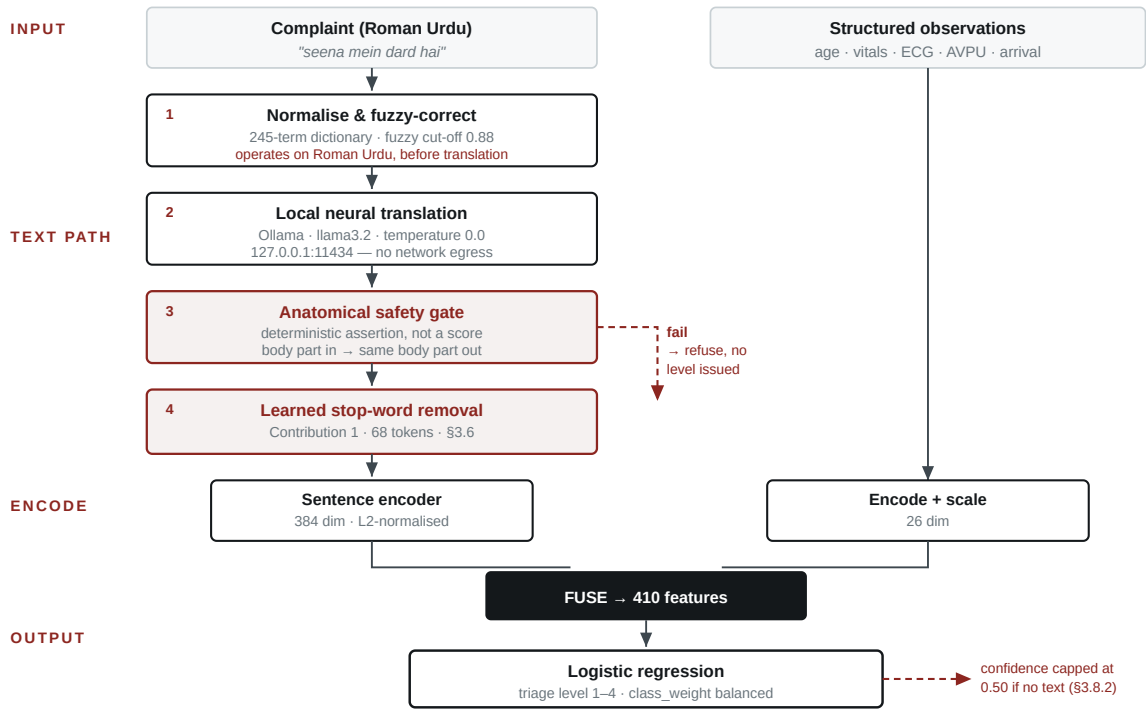


Figure 1. The full inference path. Solid arrows carry data; dashed red arrows are refusal or suppression routes. One ordering detail is easy to get wrong: normalisation and fuzzy correction operate on the Roman Urdu text *before* translation, not on the English text afterwards. The structured observations bypass the text path entirely; a complaint that fails the safety gate stops the prediction outright rather than falling back to vitals alone, because a triage level derived from vitals with the complaint silently discarded would look identical to a normal prediction.

3.2 Dataset

The model was trained on `cardiac_english_2252.csv`: 2,252 records, four triage classes, SHA-256 `42ab09e2...a728cb1c`. That hash is stamped into the model bundle at training time, so any deployed model can be traced back to the exact bytes it was trained on.

3.2.1 The data is synthetic

We state this plainly because it constrains every claim in the results section. The records were produced by `generate_cardiac_dataset.py` with seed 20260816. They are not real patient records and must not be described as such in any publication or deployment.

The generator does not sample features and then derive a label from them, which would make the label a deterministic function of the inputs and the learning problem trivial. It samples in the opposite direction: the triage level is drawn first from a fixed level mixture, and the complaint text, ECG status and vital signs are then drawn conditional on that level, with deliberate overlap between adjacent levels. A patient at Level 2 and a patient at Level 3 can present with the same complaint and similar vitals. No single feature determines the label, which is what makes the ceiling on accuracy meaningful rather than an artefact.

The complaint phrasing is not invented from nothing either. The generator draws on a phrase bank derived from the cardiac subset of a 7,855-row corpus containing 887 distinct complaints over a 636-word vocabulary; the 22 genuine Level-4 complaints in that corpus seed the Level-4 branch. The surface language is therefore realistic even though the records are not.

3.2.2 Scope

The training data contains cardiac presentations only. Trauma, burns, neurological, obstetric and paediatric complaints are out of scope, and the model bundle carries this restriction as a machine-readable field so that downstream code can check it. This is a real limitation, not a formality: a model trained exclusively on chest pain and palpitations will still emit a confident triage level for a burn, and that number will be meaningless.

Table 1. Training corpus and split.

Property	Value
Source file	<code>cardiac_english_2252.csv</code>
Records	2,252
Triage classes	4 (1 = most urgent ... 4 = least urgent)
Provenance	Synthetic — generator seed 20260816, 16 Aug 2026
Clinical scope	Cardiac complaints only
Train / test split	1,801 / 451 (80 / 20, stratified)
Random state	42
Integrity	SHA-256 recorded in the model bundle

3.2.3 How the English text was obtained

The deployed pipeline translates Roman Urdu at inference time using a language model running locally. The *training* text was not produced that way: it was translated by `gpt-4o-mini` through a hosted API. This asymmetry is discussed in §3.10 because it is a genuine threat to validity and we would rather raise it ourselves than have a reader find it in the manifest.

WHY 2,252 ROWS AND NOT 10,000

The full corpus contains 10,000 records. Only 2,252 of them — 22.5% — were translated before the translation API exhausted its credits. Rather than train on a mixture of translated and untranslated text, we trained on the translated subset alone. The consequence is that this model's accuracy is **not comparable** to the earlier 10,000-row Roman Urdu model, and the two should never be placed in the same table without this caveat attached.

3.3 Text normalisation and fuzzy correction

Roman Urdu has no orthographic standard. The same word reaches the system spelled several ways depending on who typed it — *seena*, *sina*, *seenaa* for the chest; *dard*, *dárd*, *darrrd* for pain. Before any statistical component sees the text, these variants have to be collapsed onto a single form, or the vocabulary fragments and every downstream count is computed over noise.

Normalisation runs as an ordered sequence, and it is important to be precise about *where* in the pipeline it sits:

```
Roman Urdu in → clean → rule replace → fuzzy match → diacritize → translate
```

This chain operates on the Roman Urdu complaint **before** it reaches the translator, not on the English text that comes out. Collapsing spelling variants is what allows the anatomical gate to compare source and translation against a single canonical dictionary entry, and it gives the language model a cleaner string to work from.

THE DEPLOYED ENGLISH BUNDLE SETS SKIP_NORMALIZATION

A consequence worth stating rather than leaving implicit. Because the classifier was trained on English text, the bundle carries `skip_normalization: true`, and the Roman Urdu normaliser is therefore *not* applied a second time to the English translation when features are built. That is deliberate: pushing "chest pain" back through a Roman Urdu dictionary yields "sēna dárd", which is nothing like what the model was fitted on. Serving honours the same flag as training, so the two agree.

The practical reading is that the dictionary and the fuzzy matcher are **translation-stage components** in the deployed system. They shape what the translator sees and what the safety gate can check. They do not touch the feature vector directly.

The rule-replace step applies a curated dictionary of 245 terms, organised into five groups: 39 Roman Urdu anatomy terms, 21 symptom terms, 49 additional clinical terms, 36 explicitly non-cardiac terms used for scope detection, and 100 English terms. Variants are mapped to a canonical form by grouping on their English gloss, so *seena*, *sina* and *seenaa* all resolve to one token.

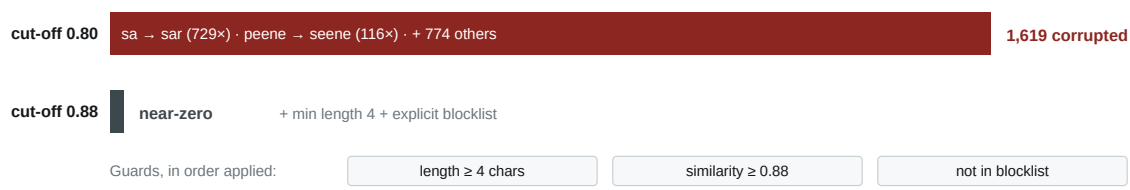
3.3.1 Choosing the fuzzy threshold

A curated dictionary cannot anticipate every misspelling, so unmatched tokens are passed to an approximate string matcher. This step is the most dangerous one in the whole preprocessing chain, because a fuzzy matcher that is slightly too permissive does not fail loudly — it silently rewrites the patient's words into different words and everything downstream proceeds as though nothing happened.

We set the similarity cut-off empirically rather than by intuition. At a cut-off of 0.80, a value that looks unremarkable, the matcher corrupted **1,619 token occurrences** across the corpus. Two failures dominated: the short token *sa* was rewritten to *sar* (head) 729 times, and *peene* (to drink) was rewritten to *seene* (chest) 116 times. The second is the more alarming of the two — it moves a complaint from the digestive tract to the chest, which is precisely the axis this system is meant to be sensitive to.

The deployed configuration raises the cut-off to **0.88**, refuses to fuzzy-match tokens shorter than four characters, and maintains an explicit blocklist of pairs that remain dangerously similar above the threshold. The first two constraints alone eliminate both failures above: *sa* is too short to be considered, and *peene/seene* falls below 0.88.

Effect of the fuzzy-match cut-off on token corruption



A token must clear all three guards to be rewritten. Anything that fails is left exactly as the patient typed it — an unrecognised token is a smaller problem than a confidently mis-corrected one.

Figure 2. Fuzzy matching is constrained by three independent guards rather than a single similarity threshold. The corruption counts at 0.80 were measured on the full corpus, not estimated.

3.4 Translation into English

The classifier operates on English. Roman Urdu complaints are therefore translated first, by a language model running on the same machine through a local HTTP service bound to `127.0.0.1:11434`. The default model is `llama3.2`, sampled at temperature 0.0 so that the same complaint yields the same translation on every run — a property the regression tests depend on. Translation is the slowest stage in the pipeline at roughly eleven seconds per complaint, and it accounts for almost all of the end-to-end latency.

3.4.1 Refusals, and the prompt structure that resolved them

An unexpected obstacle: general-purpose instruction-tuned models frequently *refused* to translate cardiac complaints. Presented with a sentence describing chest pain, the model would decline and advise the user to seek medical attention. This is a reasonable safety behaviour in a consumer chat context and a fatal one in a translation component.

Two changes fixed it. The complaint is wrapped in an explicit record delimiter, `<record>...</record>`, which frames the input as a data field to be transformed rather than a message to be answered. The conversation is then seeded with worked examples showing the model performing exactly this transformation on complaints that appear nowhere in the evaluation set — a sore throat and a swollen leg were chosen precisely because they are clinically unrelated to anything the system is tested on, so they cannot leak an answer.

The system prompt itself is substantial and structured: it establishes the operating context, the model's role, an anatomical dictionary, a section mapping kinship terms to patients (in Urdu a complaint is often reported about *ammi*, one's mother, rather than in the first person), obstetric and emergency vocabulary, symptom mappings that resolve genuine ambiguities, and phonetic variation rules.

AN APPROACH THAT FAILED

Our first attempt at handling untranslatable input was to instruct the model to emit the token `UNCLEAR` when it could not produce a faithful translation. This backfired in two directions at once. The retry logic overrode the marker, so it rarely survived; and where it did survive the model had begun parroting the few-shot examples, returning "Leg swelling" for unrelated inputs, while simultaneously refusing a perfectly ordinary complaint of *bukhar* (fever).

We removed the instruction entirely. The lesson we took from it is that asking a language model to police its own reliability puts the check inside the component being checked. Every subsequent guard in this system is deterministic code sitting *outside* the model, which is the subject of the next section.

3.5 The anatomical safety gate

Translation can fail silently. The model returns fluent, plausible English that describes a different complaint from the one the patient reported — most consequentially, a complaint about a different part of the body. Since the triage level depends heavily on which body part is involved, an undetected substitution here corrupts the prediction while leaving every visible signal looking healthy.

3.5.1 Semantic similarity was measured, and rejected

The natural approach is to embed the source and the translation in a shared multilingual space and require their cosine similarity to exceed a threshold. We implemented this, and then measured whether the threshold could actually separate good translations from bad ones. It could not.

A correct translation of a chest-pain complaint scored **0.8054**. The translation "My leg is broken" — wrong body part, wrong injury, wrong mechanism, wrong triage level — scored **0.7922** against the same source. The gap is 0.013. No threshold placed anywhere in that region separates the two classes, and one placed tightly enough to catch the failure rejects a large share of correct translations along with it.

Why the similarity threshold was abandoned

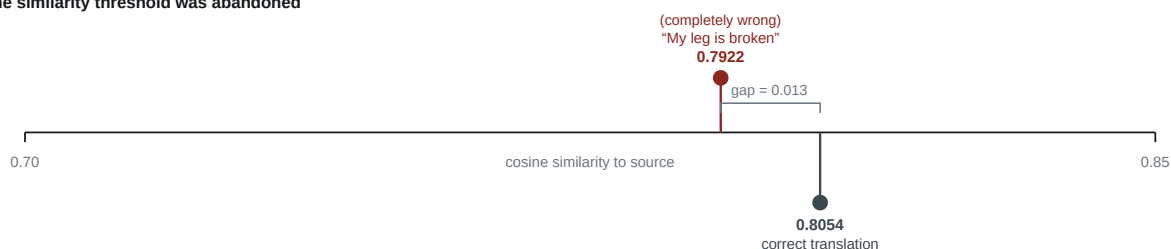


Figure 3. The two cases a similarity gate must separate, plotted on the same axis. A correct translation and a completely incorrect one are 0.013 apart. This measurement, not a design preference, is why the gate is deterministic.

3.5.2 A deterministic assertion instead

The deployed gate makes no numerical judgement. It asserts a property: **if the source complaint names a body part, the English translation must name the corresponding body part**. The mapping between them comes from the curated anatomy dictionary, so both sides of the assertion are explicit and auditable. The gate runs in both directions — it also checks that the translation has not *introduced* anatomy absent from the source, which catches the model hallucinating a location.

Matching is on word boundaries. This detail is not incidental. An earlier implementation used substring matching and failed in both directions simultaneously: it accepted "Patient feels warm" as a valid translation of a complaint about the arm (*baazu*), because "arm" occurs inside "warm"; and it rejected the entirely correct "Feeling warm and feverish" for the same reason. A single regression test would have caught neither, because the bug produced a false accept and a false reject that looked like unrelated problems.

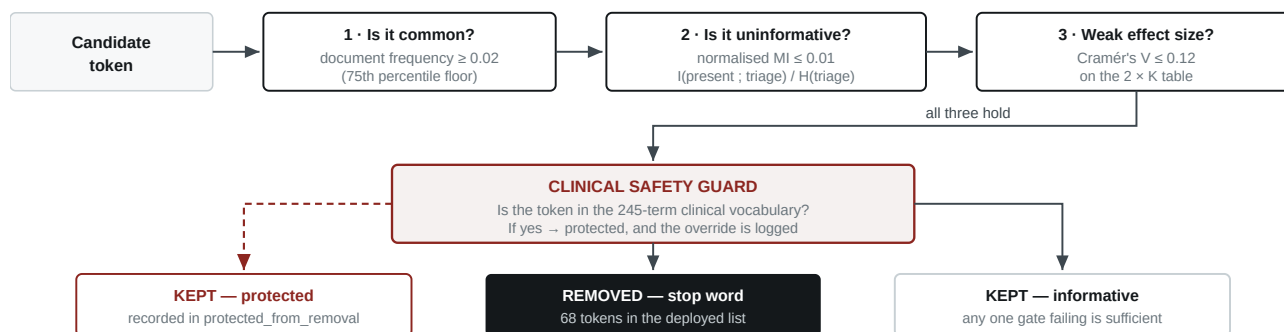
When the assertion fails, the system produces no triage level. It reports which body part was expected, which was found, and that no prediction was made.

3.6 Contribution 1: learned stop-word removal

Roman Urdu has no published stop-word list. The obvious response — writing one by hand, listing *hai*, *hain*, *ke*, *aur* and so on — has two problems. It encodes the author's intuitions about a language in which those intuitions may be wrong, and it cannot be reproduced or challenged by anyone else. We therefore learn the list from labelled training complaints using two transparent statistics and three explicit cut-offs.

3.6.1 The decision rule

A token is classified as a stop word when all three conditions hold simultaneously:



Corpus: 2,252 documents · 625 unique tokens · 113 reached testing (the rest fell below the frequency floor) · 68 removed.
 χ^2 and its p-value are computed and recorded per token for transparency, but take no part in the decision — see §3.6.2.

Figure 4. The stop-word decision rule. Three statistical gates in conjunction, followed by a clinical override that can only ever keep a token, never remove one.

3.6.2 Why effect size rather than significance

The rule originally used a chi-square test with the conventional $\alpha = 0.05$. It did not survive contact with a larger corpus, and the reason is a standard property of significance testing that is easy to forget in applied work: **p-values shrink as sample size grows**. On a corpus of a few hundred documents the test behaved sensibly. On several thousand, it began flagging associations of negligible magnitude as statistically significant, and the significance requirement then *vetoed* the removal of genuine filler — *hai* and *mein* among them — because their vanishingly small association with the label was nonetheless detectable at that sample size.

We replaced the p-value with Cramér's V, the chi-square-based effect size of the same contingency table. V measures the strength of an association rather than our confidence that it is non-zero, and it does not drift with corpus size. The learned list is consequently stable when the dataset grows, which is the property we actually needed. The chi-square statistic and its p-value are still computed and written to the report for every token, so a reader can inspect them; they simply do not participate in the decision.

3.6.3 The clinical safety guard

A purely statistical rule has one failure mode that is unacceptable in this domain. On an imbalanced corpus, a genuine symptom word can look uninformative simply because it occurs at every triage level — *dard* (pain) is present in Level 1 and Level 4 complaints alike, so knowing that it appears tells you comparatively little about the level. The statistics would therefore delete the word "pain" from a triage system.

Tokens in the 245-term clinical vocabulary are exempt from removal. Critically, every protected token that the statistics *would* have removed is recorded in the output under `protected_from_removal`. The guard is visible

rather than hidden, and it can be disabled for ablation. Note that the guard is asymmetric by construction: it can only prevent a removal, never cause one, so it cannot introduce a failure of its own.

3.6.4 Avoiding label leakage

The learner consumes the triage labels. Learning the list on all 2,252 records and then evaluating on a held-out split would leak test labels into preprocessing and inflate the reported result. The training script therefore learns stop words **on the training split alone** for every evaluation figure in this chapter, and re-learns them on the full dataset only afterwards, when producing the shipped artifact. Both lists are written out so the difference between them can be inspected.

3.6.5 Where the removal applies

The sentence encoder is the only text feature component. Learned stop-word removal applies to its English input before embedding.

Stop-word removal affected **2,208 of 2,252 complaints** — nearly the whole corpus — so this is not a marginal transformation applied to a handful of records.

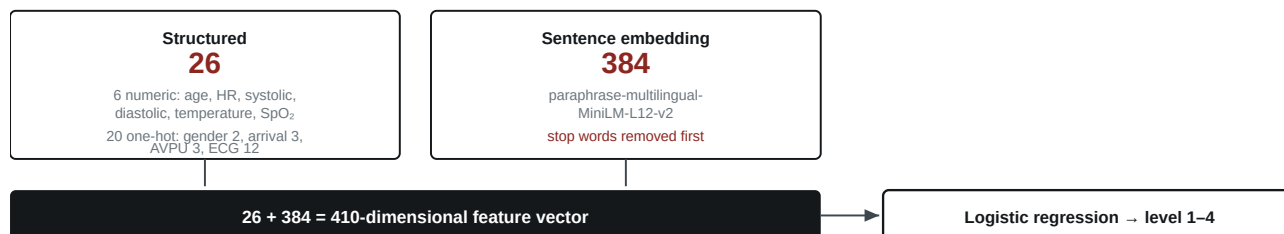
Table 2. Learned stop-word configuration and outcome.

Parameter	Value
Minimum document frequency	0.02
Document-frequency percentile	75th
Normalised mutual information threshold	≤ 0.01
Cramér's V threshold	≤ 0.12
α (recorded, not used in the decision)	0.05
Clinical terms protected	Yes (245-term vocabulary)
Documents in corpus	2,252
Unique tokens	625
Tokens reaching the statistical test	113
Stop words learned	68

3.7 Feature representation and fusion

The classifier receives a single 410-dimensional vector assembled from two blocks: 26 structured features and a 384-dimensional sentence embedding.

Feature assembly · two blocks, scaled, then concatenated



The embedding model is multilingual and CPU-resident: no GPU is required at training or inference. Its 384 dimensions are a property of the published checkpoint, not a tuned hyper-parameter. With one text block there is no cross-block scale competition.

Figure 5. Feature assembly: 26 structured features followed by 384 embedding dimensions, giving 410 classifier inputs.

One further implementation note worth recording, because it cost real debugging time. Two different sentence encoders appear in this project, and both emit 384-dimensional vectors. Mixing them therefore produces no shape error, no exception, and no warning — only quietly degraded predictions. The encoder identity is now written into the model manifest and checked at load time, because dimensional compatibility turned out to be an unreliable proxy for actual compatibility.

3.8 Classifier and training protocol

The classifier is multinomial logistic regression. This is a deliberate choice rather than a default. The feature vector already contains a 384-dimensional neural sentence representation, so the model's capacity to represent language does not depend on the classifier being deep; and a linear model over named features permits the per-feature attribution the interface displays. In a decision-support tool a clinician may override, a coefficient that can be read is worth more than a marginal accuracy gain that cannot.

The classifier is fitted with the `lbfgs` solver, `max_iter=1200`, default L2 regularisation at `C=1.0`, and — importantly — `class_weight="balanced"`. The class weighting is not incidental. The corpus is markedly imbalanced: Level 4 accounts for 20 of 451 test records against Level 2's 177. Without reweighting, a classifier minimising overall error has little incentive to predict Level 4 at all. The balanced weighting is what keeps the smallest class represented in the model's output, and it is part of why the Level-4 errors that remain are escalations to Level 3 rather than the class being ignored outright.

Data is split 80/20, **stratified by triage level**, with `random_state=42` — giving 1,801 training and 451 test records. Stratification matters here for the same reason the class weighting does: on an unstratified split, 20 Level-4 records could distribute unevenly enough to leave the test set with almost none. Every figure reported in §3.10 comes from that held-out set.

3.8.1 Metrics, and why accuracy alone is inadequate

Accuracy treats every error identically, and in triage the errors are not identical. We therefore report two directional error rates alongside it:

- **Under-triage** — the model predicts a *less* urgent level than the true one. A Level-1 patient is called Level 3 and waits. This is the dangerous direction.
- **Over-triage** — the model predicts a *more* urgent level than the true one. A Level-3 patient is seen immediately. This consumes resources but does not endanger the patient in front of you.

The two are not weighted into a single score, because doing so would require committing to an exchange rate between a delayed cardiac patient and a wasted resuscitation bay, and that is a clinical and institutional judgement rather than an engineering one. Both numbers are reported and left separate.

3.8.2 Confidence is capped when the complaint carries no text

The classifier will produce a triage level even when the complaint field is empty or nothing survives cleaning. In that situation the text blocks contribute nothing meaningful, the prediction rests on vitals alone, and — left untreated — the system reports it with the same confidence as any other. A blank complaint column in a batch file therefore used to yield a confident discharge-tier answer.

When no tokens survive cleaning, reported confidence is capped at **0.50** and an explicit warning is attached stating that the level came from vitals alone. The cap applies only on that path; ordinary predictions are reported uncapped.

We chose to flag and cap rather than to fall back to a vitals-only model, and the reasoning is worth recording. There is no vitals-only classifier in the bundle, and producing one would mean either training a second model or feeding the deployed one a zeroed text block. A zeroed block is an input the model never encountered in training, so it would answer just as confidently from a basis it does not have — trading a visible failure for an invisible one. The cap returns what the model actually said, declines to dress it up, and makes the reason impossible to miss.

This guard is deliberately narrow, and it is important not to overstate it. It detects the *absence* of tokens, not the absence of meaning. Gibberish such as "asdkfj qwoeir" survives cleaning as ordinary tokens and still produces a fully confident answer. That is a known limitation rather than an oversight; no token count can detect it, and we note it again in §3.10.1.

3.9 Embedding ablation study

Two embedding configurations are retained from the recorded experiment. B embeds raw English text. C removes learned stop words from English before embedding. Both concatenate the same structured features and use Logistic Regression on the same split of 1,801 training and 451 test records.

Table 3. Recorded embedding comparison.

Configuration	Accuracy	Under-triage	Over-triage	Features
B: raw embeddings	80.71%	13.30%	5.99%	410
C: preprocessed embeddings (deployed)	80.49%	12.64%	6.87%	410

Preprocessing reduced recorded under-triage by 0.66 percentage points and reduced accuracy by 0.22 points. These results do not establish an accuracy improvement or statistical significance. The existing C model weights are retained; removing obsolete project components did not rerun this experiment.

3.10 Results and limitations

The deployed model achieves **80.49% accuracy** on 451 held-out records, with **12.64% under-triage** and **6.87% over-triage**. Macro-averaged F1 is 76.5%. The gap between macro-F1 and accuracy remains, and is still attributable to Level 4, but it has narrowed substantially — see Table 4.

Confusion matrix · 451 held-out records · deployed configuration C

Rows = true level, columns = predicted level. Diagonal is correct. Above the diagonal is under-triage; below it is over-triage.

		PREDICTED				support	recall
		L1	L2	L3	L4		
TRUE	L1	115	18	1	1	135	85.2%
	L2	7	147	22	1	177	83.1%
	L3	0	20	85	14	119	71.4%
	L4	0	0	4	16	20	80.0%
		correct — 363				under-triage — 57 cases (12.64%)	
						over-triage — 31 (6.87%)	

The lower-left corner is still empty: no Level-3 or Level-4 patient was called Level 1, and no Level-4 patient was called Level 2. All 31 over-triage errors remain exactly one level. Under-triage: 54 by one level, 2 by two, 1 by three.

The outlined cell — one Level-1 patient classified Level 4 — is the single worst error that this confusion matrix can express. The recorded model still makes this error.

Figure 6. Confusion matrix for the deployed configuration on the held-out test set. Over-triage is uniformly one level; under-triage is not, and still includes one case spanning three levels.

Table 4. Per-class performance. Level 4 is reported despite its small support precisely because omitting it would overstate the system.

Triage level	Support	Precision	Recall	F1
Level 1 — most urgent	135	94.3%	85.2%	89.5%
Level 2	177	79.5%	83.1%	81.2%
Level 3	119	75.9%	71.4%	73.6%
Level 4 — least urgent	20	50.0%	80.0%	61.5%
Macro average	451	74.9%	79.9%	76.5%

Level 1 — the class where a mistake is most costly — is the strongest at 89.5% F1, with 94.3% precision. Level 4 remains the weakest at 61.5%, but its recall of 80.0% means the model now identifies 16 of 20 genuine Level-4 patients rather than 9. Its precision of 50.0% is the trade: the model reaches Level 4 more readily and is wrong half the time it does so, which in this direction means a low-urgency patient occasionally displacing another low-urgency patient. On a support of 20 the class is still too small to characterise confidently, and claims about Level-4 performance should be treated as indicative.

The two error directions behave differently, and the difference favours the system. Over-triage is uniformly mild: all 31 cases are wrong by exactly one level, so no patient was ever escalated more than one tier beyond their true urgency. Under-triage is less well behaved — 54 cases miss by one level, 2 by two, and one by three. That last case is a Level-1 patient classified Level 4, and it deserves to be named rather than absorbed into an aggregate: it is a maximally urgent patient assigned to the least urgent queue, the worst single error this matrix is capable of expressing. This error remains in the recorded confusion matrix. One case in 451 is a rate of 0.22%, but the appropriate response to it is not a percentage.

3.10.1 Limitations

Under-triage exceeds the intended safety band

At 12.64%, under-triage exceeds the project's intended <10% band. The model remains a research prototype and these synthetic-data results do not establish clinical safety. Comparisons across different datasets cannot isolate the effect of training-set size.

Training and serving translate differently

Training text was translated by `gpt-4o-mini` via a hosted API; serving translates locally with `llama3.2`. The model has therefore learned the vocabulary and phrasing habits of one translator and is served text from another. Any systematic divergence between them is a distribution shift the reported test accuracy does not capture, because the test set was translated by the same model as the training set. This is, in our assessment, the most serious unquantified risk in the system, and measuring it requires a parallel corpus translated both ways — work we have not done.

Synthetic data

Every number in this chapter is measured on synthetic records. They tell you the pipeline learns the structure the generator encoded. They do not tell you how the system performs on real patients, and no claim in that direction is made.

Narrow clinical scope

Cardiac presentations only. The system will return a triage level for a burn or a fracture, and that number carries no evidential weight.

Meaningless input is not detected

The no-text guard described in §3.8.2 catches an empty complaint, but nothing catches a complaint that is present and meaningless. A string of keyboard noise survives cleaning as valid tokens, passes the anatomical gate (which only constrains complaints that name a body part), and receives a normal triage level at full confidence. Detecting this reliably would require a language-model judgement about whether a complaint is a complaint, which is precisely the kind of self-assessment §3.4 gives our reasons for distrusting. It remains open.

Small and imbalanced test set

451 records, with Level 4 represented by 20. Confidence intervals on the per-class figures are wide and we have not computed them; the per-class table should be read as indicative.

INTENDED USE

This is a research prototype. It has not been clinically validated, it is not a medical device, and no output should be acted upon without a qualified clinician. The system's ability to refuse — described in §3.4 and §3.5 — exists because a decision-support tool that never declines to answer invites exactly the over-reliance this disclaimer is meant to prevent.

3.11 Reproducibility

Several measures are in place so that the results above can be reproduced and so that a deployed model can be traced to what produced it.

Data provenance is carried inside the artifact. The training file's SHA-256, row count, class count, generator name, generator version and seed are all written into the model bundle at training time, together with the synthetic-data disclosure. A bundle cannot become separated from the statement of what it was trained on.

Dependencies are pinned, for two different reasons. `scikit-learn` is pinned at 1.9.0 because the model bundle is a pickle: loading it under a different version emits a version-inconsistency warning and can silently alter predictions. That pin is load-bearing, and the correct response to a version change is retraining, not suppressing the warning. `sentence-transformers` is pinned at 6.0.1 for a weaker reason — the encoder is constructed from source each run rather than unpickled — but an unpinned upgrade of that package is what broke this environment during development, taking its own dependencies with it. Those transitive dependencies are now named explicitly in the requirements file, because a requirements file that does not name a dependency cannot restore it.

Randomness is fixed. The split seed is 42, the dataset generator seed is 20260816, and translation runs at temperature 0.0 so the same complaint produces the same English on every run.

The system is tested, not merely described. Two automated batteries ship with the project: one exercises the translation path, the safety gate, the dictionary and the normalisation chain; the other constructs the real interface and drives every tab. Both must pass on a new installation, and "correctly installed" is defined as both passing rather than as the application starting.

Evaluation caveat: structured preprocessing is fitted before the split, and the saved full-data stop-word list differs from the training-only list used for the recorded evaluation. Every stage is inspectable at run time. The interface exposes the raw input, the normalised input, the translation, the token string after stop-word removal, the encoder identity, the embedding dimension and the model bundle in use — and exports all of them to CSV. A disputed prediction can be traced through the pipeline without re-running anything.

Artifact. Deployed bundle `trriage_model_embedding_english/`, trained 9 September 2026 by `train_embedding_pipeline.py --deploy C`. Configuration C (embeddings + preprocessing), 410 features, two blocks. Dataset `cardiac_english_2252.csv`, SHA-256 `42ab09e252db94b8f7b0dea38fae47556f8d38c380fdc43fdbb37f2ea728cb1c`. Full metrics in `trriage_metrics.json`; configuration and provenance in `model_manifest.json`; the learned stop-word list, its per-token statistics and the protected-token log in `learned_stopwords.json`.

Synthetic data notice. All records used in this work were machine-generated. They are not real patient data and must not be described as such.